

CertManager — Manual de aprovisionamiento y certificados (para el equipo)

Documento: Despliegue en 443/TLS con el wildcard `*.claro.com.do` y dónde colocar el certificado en cada modo (Linux, Docker, Kubernetes). **Versión:** 1.0

Para el equipo de infraestructura/plataforma. CertManager **no es local**: se expone en el **FQDN por 443 (HTTPS)** con redirección desde 80, su servicio de **gunicorn** y su **daemon** (scheduler). La base de datos **MySQL es externa** (la provee Claro). Windows es solo para **pruebas locales**, no producción.

1. Resumen de puertos y procesos

Pieza	Detalle
Ingress / NGINX	443 (HTTPS) con el wildcard <code>*.claro.com.do</code> ; 80 → 443 (redirige)
App (gunicorn)	escucha en <code>127.0.0.1:8000</code> (interno); NGINX/Ingress hace proxy
Daemon (scheduler)	proceso aparte (<code>manage.py run_scheduler</code>): chequeos, reportes, backups
Base de datos	MySQL 8 externa (vars <code>DB_*</code> en el <code>.env</code> / ConfigMap+Secret)
Healthcheck	<code>GET /health/</code> (200 si BD ok)

El FQDN sale de `ALLOWED_HOSTS` (Linux/Docker, en `CLARO_NECESIDAD/.env`) o del ConfigMap/Ingress (Kubernetes). Por defecto: `certmanager.claro.com.do` (ajustar).

Ventana horaria de chequeo (importante para el daemon)

El chequeo masivo abre una **conexión TLS a cada host monitoreado**. Con cientos de certificados eso es carga que **conviene correr en horario valle** (madrugada), para no competir con el tráfico productivo ni generar ruido en los sistemas vigilados (IPS/WAF/logs de los hosts destino).

- **Configuración** → **Monitoreo** → “**Ventana horaria**” define esa franja. Por defecto **02:00–05:00** (zona horaria del aplicativo, `America/Santo_Domingo`).
- Con ventana definida, el daemon (`run_scheduler`) agenda el chequeo **una vez al día a la hora de inicio** (p.ej. 02:00). El rango (–05:00) comunica la franja válida; el chequeo completo debe caber dentro (con cientos de certs tarda pocos minutos a ~media hora según timeouts).
- **Vacía** = sin ventana → corre por **intervalo** (cada 24h desde que arranca el proceso), menos predecible.
- **Recomendación:** dejar 02:00–05:00 (o el horario valle de Claro). Cambiarla requiere **reiniciar** `run_scheduler` (la lee al arrancar).

2. El certificado *.claro.com.do

Se necesitan **dos archivos PEM**:

Archivo	Contenido
claro-wildcard.crt	certificado + cadena intermedia (fullchain)
claro-wildcard.key	clave privada

El wildcard cubre `certmanager.claro.com.do` y cualquier otro subdominio. **La clave privada nunca se versiona** (el repo es público; `tls/`, `/*.key`, `/*.crt` están en `.gitignore`).

Dónde colocarlo según el modo

Modo	Dónde va el certificado
Linux (systemd)	<code>/etc/ssl/claro/claro-wildcard.crt</code> y <code>/etc/ssl/claro/claro-wildcard.key</code> (o exporta <code>TLS_CERT</code> / <code>TLS_KEY</code> antes de correr <code>install_server.sh</code>).
Docker	carpeta <code>./tls/</code> en la raíz del repo: <code>tls/claro-wildcard.crt</code> y <code>tls/claro-wildcard.key</code> (la monta el contenedor NGINX).
Kubernetes	como Secret TLS <code>certmanager-tls</code> (ver §5).

3. Linux como servicio (systemd + gunicorn + NGINX)

```
# 1) cert wildcard en su sitio:
sudo mkdir -p /etc/ssl/claro
sudo cp claro-wildcard.crt /etc/ssl/claro/
sudo cp claro-wildcard.key /etc/ssl/claro/ && sudo chmod 600 /etc/ssl/claro/claro-wildcard.*

# 2) configurar el entorno (BD externa de Claro, FQDN, secret key):
cp CLARO_NECESIDAD/.env.example CLARO_NECESIDAD/.env # completar DB_*, ALLOWED_HOSTS, DJANGO_SECRET_KEY

# 3) instalar:
sudo ./install_server.sh
```

El script instala dependencias, crea el venv, migra, y deja **dos servicios systemd + NGINX** (443 + redirección 80→443):

- `certmanager.service` → gunicorn (`config.wsgi:application` , `127.0.0.1:8000`)
- `certmanager-scheduler.service` → `manage.py run_scheduler`

```
systemctl status certmanager certmanager-scheduler
curl -fsS https://certmanager.claro.com.do/health/
```

4. Docker (un solo contenedor del app + NGINX TLS)

```
# 1) cert wildcard en ./tls/
mkdir -p tls && cp claro-wildcard.crt claro-wildcard.key tls/

# 2) .env (BD externa, FQDN, secret key)
cp CLARO_NECESIDAD/.env.example CLARO_NECESIDAD/.env # completar

# 3) levantar (web + scheduler + nginx con TLS en 443)
./install_docker.sh
curl -fsS https://certmanager.claro.com.do/health/
```

`docker-compose.app.yml` levanta `nginx` (80/443), `web` (gunicorn interno) y `scheduler`. **No** levanta BD (es externa).

5. Kubernetes

Manifiestos en `k8s/` (Deployments `web/scheduler`, Service, Ingress 443, Job de migración). El certificado va como **Secret TLS**:

```
kubectl -n certmanager create secret tls certmanager-tls \
  --cert=claro-wildcard.crt --key=claro-wildcard.key
```

Pasos completos (imagen, ConfigMap, Secret de app, Ingress, bootstrap) en `k8s/README.md`.

El **scheduler corre con 1 réplica** (el lock es por-proceso). Revisar con plataforma el `ingressClassName`, el registro de imágenes y el gestor de secretos.

6. Datos: migración del monitoreo (`cert.txt`)

La carga del `cert.txt` **es la migración de todo el monitoreo** (dominios, umbrales, correos de soporte) al aplicativo — no es un paso opcional, solo está **pendiente del `cert.txt` actualizado**. Mientras tanto se puede aprovisionar Owner + configuración con `--skip-certs` y completar la migración después:

```
# bootstrap mínimo (Owner + configuración por defecto), SIN certificados:
./data_update_certs_app.sh --skip-certs # Linux
docker compose -f docker-compose.app.yml exec web ./data_update_certs_app.sh --skip-certs

# cuando llegue el cert.txt actualizado:
# - Linux: colocarlo en la raíz y correr ./data_update_certs_app.sh
# - Docker: copiarlo al contenedor (no se hornea en la imagen) y correr:
#   docker cp cert.txt certforge-web-1:/app/cert.txt
#   docker compose -f docker-compose.app.yml exec web ./data_update_certs_app.sh
# - Kubernetes: kubectl cp cert.txt <pod-web>:/app/cert.txt y exec el comando
./data_update_certs_app.sh # carga ubicaciones, grupos sp_*, destina
```

Detalle de formato y reglas: `03_cambios_para_produccion.md` §7.

7. Checklist para el equipo

- [] Solicitar/confirmar **MySQL 8 externa** (host, BD, usuario, clave) → vars `DB_*`.
- [] Tener el **wildcard** `*.claro.com.do` (cert+cadena + clave) y colocarlo según el modo (§2).
- [] Generar `DJANGO_SECRET_KEY`, fijar `ALLOWED_HOSTS` /FQDN.
- [] Aperturas de firewall (egress/ingress) → `02_necesidades_instalacion.md`.
- [] Desplegar (Linux / Docker / K8s) y validar `GET /health/`.
- [] Bootstrap del Owner (`data_update_certs_app --skip-certs`).
- [] (Más adelante) migrar `cert.txt` actualizado.